Name of Course	**Bachelor of Computer Application (BCA)**
Semester	**V**
Name of Subject	**System Analysis and Design (SAAD)**
Subject Code	**BCA-501**
Marks	**75 Marks**
Lectures	**50 Lectures**

Unit I

1. Introduction of System Concept & System Development Life Cycle

1.1 System Concept: Definition, Characteristics
1.2 Elements of system, Physical and abstract system
1.3 Open and closed system, man-made information systems
1.4 System Development Life Cycle: Various phases of system development
1.5 Considerations for system planning and control for system success
1.6 Role of system analyst

1.1 System Concept: Definition, Characteristics:

Here's a concise explanation of the **System Concept** with **Definition** and **Characteristics**, suitable for a 5 mark answer:

Definition:

A system is a set of interrelated components working together toward a common goal by accepting inputs, processing them, and producing outputs in an organized way.

Characteristics of a System:

1. Components:

A system is made up of different parts or elements (subsystems) that interact with each other.

2. Interdependence:

All components are dependent on one another. A change in one affects the others.

3. Boundary:

Every system has a boundary that defines what is inside and outside the system.

4. Input and Output:

Systems receive inputs from the environment and produce outputs after processing.

5. Goal-oriented:

All systems are designed to achieve specific objectives or goals.

1.2 Elements of system, Physical and abstract system:

Here’s a clear and concise explanation of the **Elements of system**, **Physical** and **abstract system:**

Elements of a System:

A system is made up of several essential elements that work together to achieve a goal:

1. Input:

The data, material, or energy put into the system for processing.

2. Process (Throughput):

The activity that transforms input into output.

3. Output:

The result produced by the system (e.g., information, product).

4. Control:

Directs the system's functioning and ensures it works properly.

5. Feedback:

Information about the output used to make improvements or adjustments.

6. Environment:

Everything outside the system that affects its operation.

7. Boundary:

The limit that separates the system from its environment.

Physical and Abstract Systems:

1. Physical System:

A system that **can be touched and seen**.
It includes real components like machines, people, or equipment.

Example:

A computer, car engine, or human body.

2. Abstract System:

A system that **cannot be physically touched**.
It deals with concepts, models, or ideas.

Example:

Mathematical equations, software algorithms, or an organizational structure.

1.3 Open and closed system, man-made information systems:

Open and Closed Systems:

1. Open System:

An open system **interacts with its environment**.
It receives input from and gives output to the environment.
Most real-world systems are open.

Example:

A business organization (takes raw materials, gives out products).

2. Closed System:

A closed system **does not interact** with its environment.
It is **self-contained** and isolated from external influences.

Very rare in reality, but used in theoretical models.

Example:

A chemical reaction in a sealed container (theoretically closed).

Man-Made Information Systems:

These are **systems created by humans** to manage data and information.
They help in **processing, storing, and communicating** information to support decision-making.

Types:

1. **Transaction Processing Systems (TPS) –** handle day-to-day transactions.
2. **Management Information Systems (MIS) –** support middle management decisions.
3. **Decision Support Systems (DSS) –** assist in complex decision-making.

Example:

Banking systems, payroll systems, inventory management systems.

1.4 System Development Life Cycle: Various phases of system development:

System Development Life Cycle (SDLC):

Definition:

The **System Development Life Cycle (SDLC)** is a structured process used for developing information systems. It consists of several phases to ensure the successful creation, implementation, and maintenance of systems.

Phases of SDLC:

1. **Requirement Analysis:**
   Gather and analyze user needs.
   Understand what the system should do.

2. **System Design:**
   Create system architecture and design layouts.
   Decide hardware, software, databases, etc.

3. **Development (Coding):**
   Programmers write code based on the design.

4. **Testing:**
   Test the system for errors and bugs.
   Ensure it meets user requirements.

5. **Implementation:**
   Deploy the system into the actual working environment.

6. **Maintenance:**
   Perform regular updates, bug fixes, and enhancements.

1.5 Considerations for system planning and control for system success:

Considerations for System Planning and Control for System Success:

Proper planning and control are essential to ensure the successful development and implementation of a

system. The following points must be considered:

1. Clear Objectives:

Define specific goals the system should achieve.
Align the system's purpose with organizational needs.

2. Feasibility Study:

Analyze **technical**, **economic**, and **operational** feasibility.
Ensure the system is practical and cost-effective.

3. Resource Availability:

Plan for required **budget**, **time**, **technology**, and **skilled personnel**.
Allocate resources efficiently.

4. Risk Management:

Identify potential risks early (like system failure or delays).
Plan preventive and corrective actions.

5. Monitoring and Control:

Continuously track system progress using control techniques.
Ensure the project stays on schedule and meets quality standards.

These considerations help in minimizing failures and ensure that the system supports the organization's objectives effectively.

1.6 Role of system analyst:

Role of a System Analyst:

A **System Analyst** is a key professional responsible for analyzing, designing, and implementing information systems to solve business problems.

Roles and Responsibilities:

1. Requirement Gathering:

Interacts with users and stakeholders to understand system needs.

2. System Design:

Prepares system specifications, data flow diagrams, and design models.

3. Feasibility Study:

Evaluates the technical, economic, and operational feasibility of a proposed system.

4. Communication Bridge:

Acts as a link between management, users, and technical teams (like developers).

5. Testing and Implementation Support:

Assists in system testing, user training, and supports during implementation.

Importance:

A system analyst ensures the system is **effective, user-friendly, cost-efficient**, and meets business goals.

Unit II

2. System Planning, Feasibility study & Cost-Benefit Analysis

2.1 Basis for planning in system analysis: Dimensions of Planning
2.2 Initial Investigation: Determining user's requirements and analysis
2.3 fact finding process and techniques
2.4 Feasibility study: Technical
2.5 Operational & Economic Feasibilities
2.6 Cost/Benefit Analysis

2.1 Basis for planning in system analysis: Dimensions of Planning:

Basis for Planning in System Analysis:

Planning in system analysis lays the foundation for successful system development. It involves identifying **what** needs to be done, **how**, and **when**, based on organizational goals and system requirements.

Dimensions of Planning:

1. Scope Planning:

Defines the boundaries of the system.
Identifies what will be included and excluded.

2. Resource Planning:

Determines the resources required — **people, time, budget, tools**, etc.
Ensures efficient allocation.

3. Time Planning (Scheduling):

Estimates the **timeframe** for each phase.
Helps in setting deadlines and milestones.

4. Cost Planning:

Involves **budget estimation** and financial planning.
Ensures the system remains cost-effective.

5. Risk Planning:

Identifies potential risks and prepares strategies to **avoid or handle** them.
Improves system reliability and success.

These dimensions ensure that the system development is **well-organized, efficient**, and meets user expectations.

2.2 Initial Investigation: Determining user's requirements and analysis:

Definition:

Initial investigation is the **first step** in system analysis, where the system analyst studies the existing system, understands user needs, and identifies the problem to be solved.

Steps Involved:

1. Understanding the Problem:

Identify why a new system is needed.
Understand the current issues or limitations.

2. Identifying Objectives:

Define what users expect from the new system.
Set clear goals based on user input.

3. Fact-Finding Techniques:

Use interviews, questionnaires, observations, and document reviews to gather information from users.

4. Analyzing User Requirements:

Study the collected data to understand **what users want**, not just **what they say**.
Identify both **functional** and **non-functional** requirements.

5. Preparing a Report:

Document findings and suggest whether to proceed with the full system study.

Importance:

It ensures that the project starts with a **clear understanding of user needs**, reducing errors and rework later.

2.3 fact finding process and techniques:

Definition:

The **fact-finding process** is the method used by system analysts to gather information about the existing system and user requirements. It helps in understanding the current problems and expectations for the new system.

Fact-Finding Techniques:

1. Interviewing:

Face-to-face interaction with users to gather detailed insights.
Helps in understanding opinions and expectations.

2. Questionnaires:

Set of written questions given to users.
Useful for collecting data from a large number of people quickly.

3. Observation:

Analyst watches users perform tasks to understand how the current system works.
Reveals real-world practices and issues.

4. Document Review:

Studying existing reports, forms, manuals, and system documents.
Helps understand procedures and historical data.

5. Record Sampling:

Examining a sample of past records or transactions.
Used for analyzing trends or common issues.

Importance:

Accurate fact-finding ensures the system is built based on **real needs**, improving its success and user satisfaction.

2.4 Feasibility study: Technical:

Definition:

Technical feasibility is a part of the overall feasibility study that evaluates whether the **technology and resources** needed to develop the proposed system are available and capable of meeting the requirements.

Key Aspects of Technical Feasibility:

1. Hardware and Software Availability:

Checks if the required hardware and software are available or need to be purchased or developed.

2. Technical Expertise:

Assesses whether the team has the necessary technical skills and experience to build and maintain the system.

3. System Compatibility:

Ensures that the new system will work smoothly with the existing systems and infrastructure.

4. Scalability and Performance:

Evaluates if the system can handle future growth in data and users efficiently.

5. Time Constraints:

Determines whether the system can be developed and implemented within the required timeframe using available technology.

Conclusion:

If a system is **technically feasible**, it means it is possible to build and run it using current technology, tools, and expertise.

2.5 Operational & Economic Feasibilities:

1. Operational Feasibility:

Definition:

Operational feasibility checks whether the proposed system will **function effectively in the user environment** and if it will be accepted by users.

Key Points:

User Acceptance: Will users be comfortable and willing to use the new system?
Work Environment Fit: Can the system be integrated into existing workflows?
Training Needs: Are users ready or do they need training to operate the system?
Support and Maintenance: Are there resources available to maintain and support the system?

Conclusion:

If a system is **operationally feasible**, it means it will work practically and meet user expectations in a real-world setting.

2. Economic Feasibility:

Definition:

Economic feasibility checks whether the **benefits** of the proposed system **outweigh the costs** involved in developing and implementing it.

Key Points:

Cost Analysis: Includes development, hardware, software, training, and maintenance costs.

Benefit Analysis: Looks at savings, efficiency, profit, or service improvements.

Cost-Benefit Comparison: Compares total costs with expected financial or non-financial benefits.

Conclusion:

If a system is **economically feasible**, it is considered a good investment for the organization.

2.6 Cost/Benefit Analysis:

Definition:

Cost/Benefit Analysis (CBA) is a technique used to **compare the costs** of developing and implementing a system with the **expected benefits**, to determine whether the project is financially worthwhile.

Types of Costs:

1. Development Costs:

Expenses for software, hardware, labor, training, etc.

2. Operating Costs:

Ongoing costs like maintenance, support, upgrades, etc.

Types of Benefits:

1. Tangible Benefits:

Measurable in money (e.g., reduced labor costs, increased sales).

2. Intangible Benefits:

Not directly measurable (e.g., improved customer satisfaction, better decision-making).

Purpose:

Helps in **decision-making** by showing if benefits outweigh costs.

Supports **economic feasibility** during system planning.

Conclusion:

If the **total benefits** are greater than the **total costs**, the system is considered **cost-effective and worth implementing**.

Unit III

3. Tools of structured Analysis

3.1 Data Flow diagram
3.2 Data dictionary
3.3 IPO charts
3.4 HIPO charts
3.5 Gantt charts
3.6 Pseudo codes
3.7 Flow charts
3.8 Decision tree
3.9 Decision tables

3.1 Data Flow diagram:

Definition:

A **Data Flow Diagram (DFD)** is a graphical tool used to represent the **flow of data** in a system. It shows **how data moves** between processes, data stores, and external entities.

Purpose of DFD:

To **visually model** how data is processed in a system.
Helps analysts and developers understand the system's **inputs, processes, and outputs**.

Key Components of a DFD:

1. Processes (Circles or Ovals):
Show how data is transformed or processed.

2. Data Flows (Arrows):
Indicate the direction and movement of data between elements.

3. Data Stores (Open-ended Rectangles):
Represent places where data is stored for later use.

4. External Entities (Squares or Rectangles):
Represent outside sources or destinations of data (e.g., users, other systems).

Types of DFDs:

Level 0 (Context Diagram): Shows the entire system as a single process.
Level 1, 2, etc.: Break down the system into more detailed sub-processes.

Conclusion:

DFDs are useful for understanding and analyzing system requirements and designing efficient data processing systems.

3.2 Data dictionary:

Definition:

A **Data Dictionary** is a **centralized repository** that stores **definitions and details** about all data elements used in a system. It acts like a reference guide for developers, analysts, and users.

Purpose:

To provide **clear and consistent descriptions** of data elements.
To help in **system design, development, and maintenance**.
Ensures **standardization** of data across the system.

Contents of a Data Dictionary:

1. **Data Name –** The name of the data element.
2. **Description –** What the data represents.
3. **Type –** The data type (e.g., Integer, String, Date).
4. **Size/Length –** Maximum number of characters or digits.
5. **Allowed Values –** Valid range or options (e.g., Yes/No).
6. **Source –** Where the data comes from.
7. **Destination –** Where the data is used or stored.

Example Entry:

Data Name	Type	Size	Description	Allowed Values
Student_ID	String	10	Unique ID of student	Alphanumeric
Issue_Date	Date	------	Date of book issue	Valid date

Conclusion:

The data dictionary is essential for **avoiding confusion**, maintaining **data integrity**, and improving **system documentation**.

3.3 IPO charts:

IPO Charts (Input-Process-Output Charts):

Definition:

An **IPO Chart** is a simple tool used to represent the **Input**, **Process**, and **Output** of a system or a function. It helps in understanding how a task is performed step-by-step.

Purpose:

To **break down** a system or function into basic components.
Helps analysts and developers **design and document** systems clearly.
Useful in both **programming** and **system analysis**.

Structure of an IPO Chart:

Input	Process	Output
Data entered into system	Task or operation performed	Result or outcome produced

Example: Payroll System IPO Chart:

Input	Process	Output
Employee ID, Hours Worked, Pay Rate	Calculate salary (Hours x Rate), Apply deductions	Net Salary Slip

Conclusion:

IPO charts help in **organizing logic** clearly, making system design and documentation more

effective and understandable.

3.4 HIPO charts:

Definition:

HIPO (Hierarchy plus Input-Process-Output) charts are a **visual tool** used in system analysis and design to show a system's **structure (hierarchy)** and the **details of each function** in terms of input, process, and output.

Purpose:

To provide a **top-down view** of the system.
Breaks down the system into **modules** and **sub-modules**.
Helps in better **understanding, designing, and documenting** large systems.

Components of HIPO Charts:

1. Hierarchy Chart:

Shows the **overall structure** of the system.
Main modules at the top, sub-modules underneath.

2. IPO Charts for Each Module:

Describes **what each module does** using:

Input – What data goes in
Process – What the module does
Output – What result comes out

Example (Library System):

Hierarchy Chart:

```
Library System
    ├── Issue Book Module
    ├── Return Book Module
    └── Manage Members Module
```

IPO for "Issue Book Module":

Input	Process	Output
Book ID, Member ID	Check availability, update record	Confirmation message

Conclusion:

HIPO charts help in **breaking down complex systems**, making them easier to **design, understand, and develop**.

3.5 Gantt charts:

Definition:

A **Gantt Chart** is a type of **bar chart** used in **project management** to visually represent a project's schedule. It shows **tasks or activities** along with their **start and end dates**, making it easy to track progress over time.

Purpose:

Helps in **planning, scheduling, and monitoring** project activities.
Useful for **system development**, showing each phase of the SDLC (System Development Life Cycle).
Helps in **resource allocation** and identifying **task dependencies**.

Key Features:

1. **Tasks/Activities:** Listed on the vertical (Y) axis.
2. **Time Scale:** Shown on the horizontal (X) axis (days, weeks, etc.).
3. **Bars:** Horizontal bars show the **duration** of each task.
4. **Dependencies:** May show how one task depends on another.

Example (Mini Project Gantt Chart):

Task	Start	End	Duration
Requirement Analysis	Day 1	Day 3	3 days
System Design	Day 4	Day 6	3 days
Development	Day 7	Day 12	6 days

 Bars drawn across the timeline show when each task happens.

Conclusion:

Gantt charts provide a **clear visual timeline**, helping project managers keep the system project **on track** and well-organized.

3.6 Pseudo codes:

Definition:

Pseudocode is a **simplified, informal way of writing a program's logic** using plain English-like statements. It does not follow any specific programming language syntax but helps in planning and designing code.

Purpose:

To **outline the logic** of a program before actual coding.
Helps programmers and analysts **focus on the steps** rather than syntax.
Makes it easier to **translate logic into any programming language** later.

Key Features:

1. **Simple and readable** language.
2. **Structured like real code** with loops, conditions, and functions.
3. No strict syntax – focus is on logic, not formatting.

Example:

Task: Find the sum of two numbers

```
START
    INPUT number1, number2
    sum ← number1 + number2
    PRINT sum
END
```

Conclusion:

Pseudocode is a valuable tool in **system analysis and programming**. It simplifies complex logic,

improves communication among team members, and makes the coding process smoother and error-free.

3.7 Flow charts:

Definition:

A **flowchart** is a **diagrammatic representation** of an algorithm or process, using **symbols** to show the **flow of control** and **steps** involved in solving a problem or performing a task.

Purpose:

To visually describe **how a process or system works**.

Helps in **understanding, analyzing, and debugging** the logic before implementation.

Useful in **system design, programming, and documentation**.

Common Flowchart Symbols:

Symbol	Meaning
Rectangle	**Process/Operation** step
Diamond	**Decision/Condition** (Yes/No)
Oval	**Start/End** of the flow
Arrow	**Flow direction between steps**

Conclusion:

Flowcharts are an essential tool for **planning and explaining logic** clearly. They are easy to understand and help reduce errors during system or program development.

3.8 Decision tree:

Definition:

A **decision tree** is a **graphical tool** used to represent **decision-making logic** in a **tree-like structure**. It helps in choosing between different options based on certain conditions or rules.

Purpose:

To **visually map out choices** and their possible outcomes.

Helps in **analyzing complex decisions** with multiple conditions.

Useful in **system analysis, programming, and AI** (especially in decision support systems and data mining).

Structure of a Decision Tree:

1. Root Node:

The first decision or condition in the tree.

2. Branches:

The **possible choices** or outcomes of a condition (e.g., Yes/No, True/False).

3. Leaf Nodes:

The **final outcomes or actions** taken after following a branch.

Example: Check if a student passes:

```
[Marks >= 40?]
    /       \
```

```
        Yes       No
        /           \
[Print "Pass"]  [Print "Fail"]
```

Conclusion:

Decision trees are useful for making **logical decisions step-by-step**. They simplify complex conditions and are easy to read and understand.

3.9 Decision tables:

Definition:

A **decision table** is a **tabular method** used to represent and analyze **complex decision logic**. It lists all **possible conditions** and the corresponding **actions** to be taken based on those conditions.

Purpose:

To **systematically model** decision-making situations.
Helps in handling **multiple combinations** of conditions and outcomes.
Useful in **system analysis, design, and testing**.

Structure of a Decision Table:

Conditions	**Rule 1**	**Rule 2**	**Rule 3**	**Rule 4**
Age ≥ 18	Yes	Yes	No	No
ID Provided	Yes	No	Yes	No
Action	Issue ID	Reject	Reject	Reject

Conditions: Tests or criteria.
Rules: Different combinations of Yes/No or True/False.
Actions: What to do for each rule.

Conclusion:

Decision tables are excellent for **clarifying logic** and ensuring all possible **combinations of inputs** are considered. They reduce errors and improve decision transparency.

Unit IV

4. Form and database design

4.1 Input/ Output and Form Design
4.2 Introduction to files and database design
4.3 File structures and organization
4.4 Objectives of database design
4.5 Logical and physical view of data

4.1 Input/ Output and Form Design:

Definition:

Input/Output (I/O) and Form Design refers to the process of creating **user-friendly interfaces** for entering (input) and viewing (output) data in a system. It plays a key role in the system's **usability and efficiency**.

Input Design:

Focuses on how **data is entered** into the system.
Ensures **accuracy, completeness, and validation** of data.
Should be **simple**, **clear**, and **error-free**.

Examples:

Textboxes for names, dropdowns for options, date pickers for DOB, etc.

Output Design:

Focuses on how **information is presented** to users.
Should be **clear, well-organized, and relevant**.
Can be in the form of **reports, dashboards, printouts, or screen displays**.

Examples:

Receipts, monthly reports, search result screens, etc.

Form Design:

Combines both input and output elements.
Should follow **consistent layout**, **labels**, and **navigation**.
Improves **user experience** and **data processing**.

Conclusion:

Effective I/O and form design ensures that users can **easily interact** with the system, resulting in **better performance, accuracy**, and **user satisfaction**.

4.2 Introduction to files and database design:

Definition:

File and Database Design refers to the planning and structuring of how **data is stored, accessed, and managed** in a system. It ensures data is **organized, secure, and easily retrievable**.

Files:

A **file** is a collection of **related records** stored together.
Used in **traditional file-based systems**.

Types:

Sequential Files (read in order)
Direct Access Files (access any record quickly)

Limitations:

Data **redundancy**, **inconsistency**, and **difficult maintenance**.

Databases:

A **database** is an **organized collection of data**, managed by a **Database Management System (DBMS)**.

Supports **multiple users**, **data integrity**, and **data relationships**.

Advantages over files:

Reduced redundancy
Data sharing
Easy updating and retrieval
Security and backup

Database Design Includes:

1. **Identifying entities and relationships**
2. **Creating tables and keys (Primary, Foreign)**
3. **Normalizing data** to avoid duplication
4. Ensuring **efficient storage and access**

Conclusion:

Proper file and database design is essential for **reliable, efficient, and scalable** systems that meet user and business needs.

4.3 File structures and organization:

Definition:

File structure and organization refers to the way **data records** are stored and managed within a file. The method used affects **data access speed**, **storage efficiency**, and **system performance**.

Types of File Organization:

1. Sequential File Organization:

Records are stored **in a fixed order** (usually sorted).
Best for **batch processing**.
Example: Payroll system.
Limitation: Slow for searching and updating.

2. Direct/Random Access File Organization:

Records are stored at specific locations using a **hashing algorithm**.
Allows **quick access** to any record.
Example: Bank customer info system.

3. Indexed Sequential File Organization:

Combines **sequential** and **indexing**.
An **index** is used to quickly find the starting point, then search sequentially.
Example: Library management system.

Factors to Consider in File Organization:

Speed of access
Frequency of updates
File size
Type of application

Conclusion:

Choosing the right file organization method is essential for improving **system performance**, ensuring **quick access**, and making **data processing efficient**.

4.4 Objectives of database design:

Definition:

Database design is the process of organizing data into a structured format to ensure it is **accurate,**

efficient, secure, and **easy to maintain**.

Main Objectives:

1. Data Integrity:

Ensures **accuracy and consistency** of data across the system.
Avoids duplication and errors.

2. Data Security:

Protects data from **unauthorized access** and **loss**.
Includes user roles, permissions, and backups.

3. Minimize Data Redundancy:

Avoids storing the same data in multiple places.
Leads to efficient use of storage and fewer inconsistencies.

4. Efficient Data Access:

Speeds up **retrieval, update, and reporting**.
Optimized design leads to better **system performance**.

5. Support for Future Growth:

Design should be **scalable** to handle **more data** and **new features** in the future.

Conclusion:

The goal of database design is to create a structure that supports the system's **current and future needs**, while ensuring **efficiency, reliability, and security**.

4.5 Logical and physical view of data:

Definition:

In database systems, data can be viewed from two perspectives:

The **logical view** (what data is, and how users see it)
The **physical view** (how data is stored and organized internally)

1. Logical View:

Focuses on the **structure and meaning** of data.
Describes **what data is stored**, its **relationships**, and **constraints**.
Concerned with **tables, fields, data types**, etc.
Used by **designers, analysts, and end-users**.

Example: A user sees a table named `Students` with columns like Name, Age, and ID.

2. Physical View:

Deals with the **actual storage** of data on hardware (e.g., hard disks).
Includes **file structures**, **indexes**, **data access paths**, etc.
Handled by **database administrators (DBAs)**.
Optimized for **performance and storage efficiency**.

Example: How the `Students` table is stored in binary form in memory, or how indexes are created.

Key Difference:

Aspect	Logical View	Physical View
Focus	What data is	How data is stored
Used by	Users, designers	DBAs, system developers
Concerned with	Tables, fields, relations	Files, indexes, storage format

Conclusion:

Understanding both views is essential for **designing efficient, user-friendly, and well-performing** database systems.